Vulnerability Assessment Findings Report

Target: https://ztw.ctbb.show/#home

Date: 17/04/2025
Project: FUTURE_CS_01

Table of Contents

Confidentiality Statement 3
Disclaimer 3
Contact Information 3
Assessment Overview 3
Assessment Components 4
External Penetration Test 4
Finding Severity Ratings 4
Scope 4
Executive Summary 5
Attack Summary 5
Penetration Test Findings 6
1) IDOR — Insecure Direct Object Reference 6
2) Broken Access Control 7
3) Reflected Cross-Site Scripting (XSS) 8
4) CSRF via GET Request 9-10
Security Weakness 11

Confidentiality Statement

This document is the exclusive property of the organisation and Future Interns Cyber Security Programme. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires consent of both parties. This document may be shared with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

Disclaimer

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period. Time-limited engagements do not allow for a full evaluation of all security controls. The assessment was prioritised to identify the weakest security controls an attacker would exploit. It is recommended to conduct similar assessments on an annual basis by internal or third-party assessors to ensure the continued effectiveness of controls.

Contact Information

Name	Title	Contact Information
Target Organisation	ztw.ctbb.show	-
Web Admin	Information Security (CISO)	Email: admin@ztw.ctbb.show
Dev Lead	IT Manager	Email: dev@ztw.ctbb.show
Service	-	-
Assessor	-	-
Future Interns — CS Intern	Cyber Security Intern	Future Interns Programme — CS Track

Assessment Overview

From April 14th, 2025 to April 17th, 2025, a security assessment was conducted to evaluate the security posture of the web application at https://ztw.ctbb.show/#home compared to current industry best practices, including a passive external penetration test. All testing performed is based on the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment, OWASP Testing Guide (v4), and customised testing frameworks. Phases of penetration testing activities include the following:

- Planning – Customer goals are gathered and rules of engagement obtained.
- Discovery – Perform scanning and enumeration to identify potential vulnerabilities, weak areas, and exploits.
- Attack – Confirm potential vulnerabilities through exploitation and perform additional discovery upon new access.
- Reporting – Document all found vulnerabilities and exploits, failed attempts, and company strengths and weaknesses.

Assessment Components

External Penetration Test

An external penetration test emulates the role of an attacker attempting to gain access to the application without internal resources or inside knowledge. Open-source intelligence (OSINT) techniques were used to gather information, followed by passive scanning and enumeration to identify potential vulnerabilities including IDOR, Broken Access Control, Reflected XSS, and CSRF.

Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organisation's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Scope

Assessment	Details
External penetration test	https://ztw.ctbb.show/#home

Executive Summary

During April 14 to April 17, 2025, a security assessment was conducted on the web application https://ztw.ctbb.show/#home. Tools used for scanning include OWASP ZAP, Burp Suite, Nmap, and Browser DevTools. By leveraging a series of passive analysis techniques, four significant vulnerabilities were identified including one Critical-severity Reflected XSS that enables full account takeover. It is highly recommended that the vulnerabilities be addressed as soon as possible, as they are exploitable through basic techniques with minimal effort.

Total records generated	Critical	High Severity	Medium Severity	Low Severity
4	1	3	0	0

Attack Summary

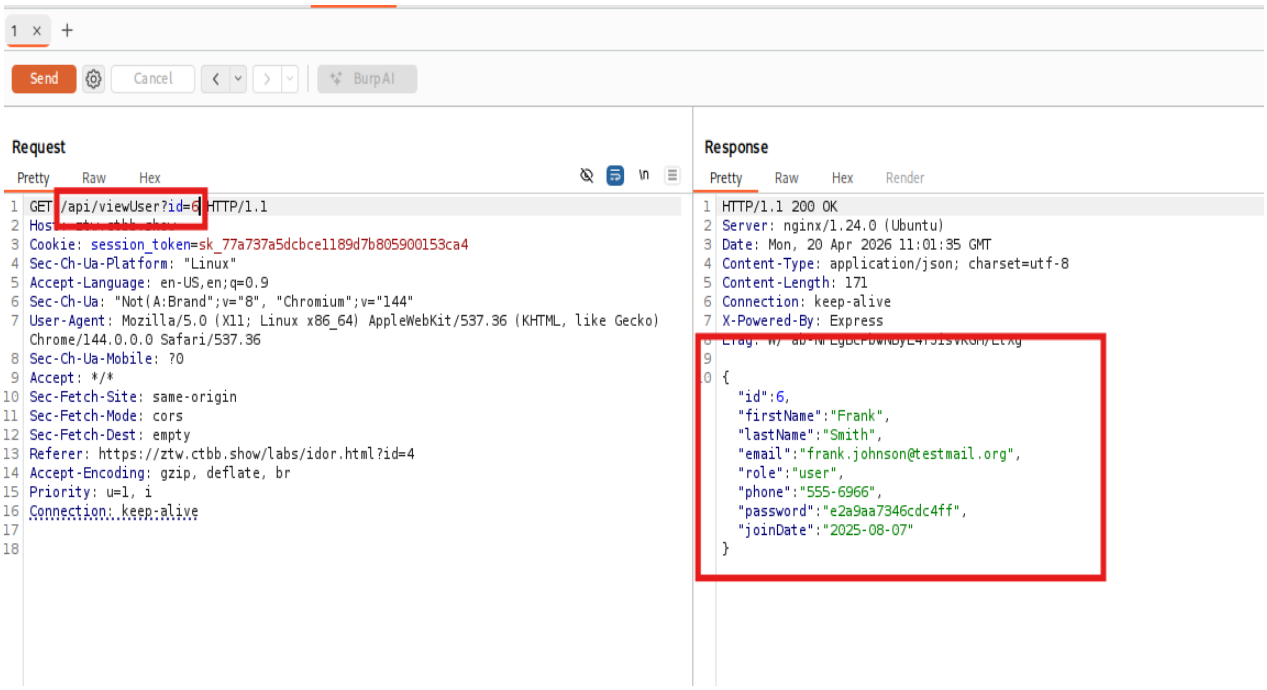
Sl. no	Action	Recommendation	Severity
1	IDOR — accessing other users' data via predictable IDs at /api/users/{id}/profile	Implement server-side object-level authorisation; replace sequential IDs with UUIDs	**High**
2	Broken Access Control — regular users reaching /admin/* routes and APIs	Enforce server-side RBAC; remove client-side role checks; deny by default	**High**
3	Reflected XSS in search/URL parameter reflected into response without encoding	Implement output encoding (DOMPurify / OWASP Encoder) and deploy Content-Security-Policy	**Critical**
4	CSRF via GET — state-changing operations (delete/reset) triggered via GET requests	Move to POST/DELETE; add CSRF tokens; set SameSite=Strict on session cookies	**High**

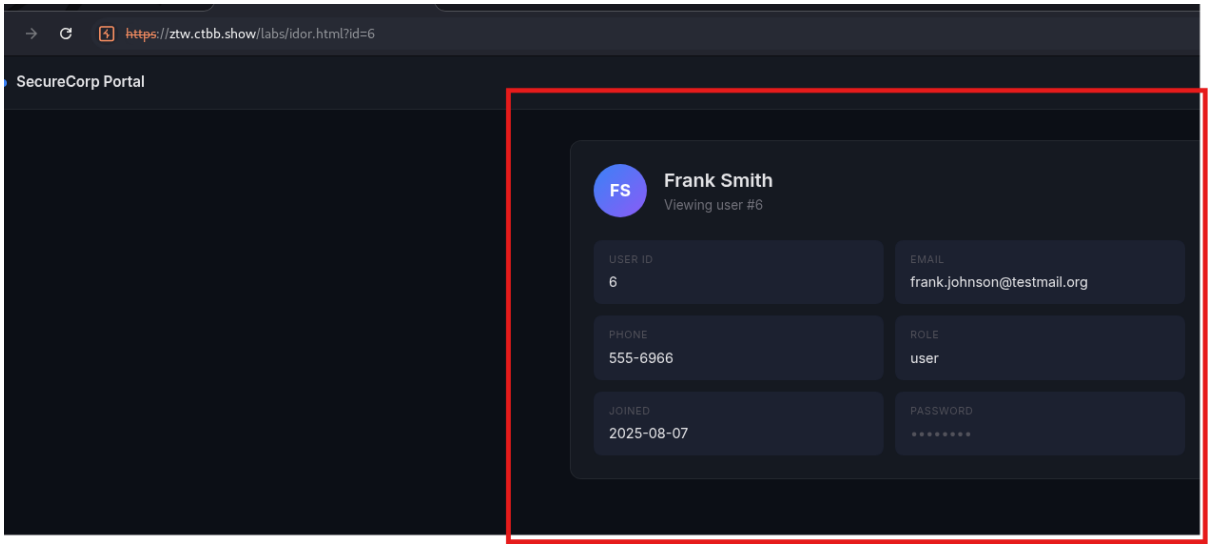
Penetration Test Findings

1) Insecure Direct Object Reference (IDOR) (High)

Description	The application exposes internal numeric object identifiers directly in URLs and API parameters (e.g. /api/users/1042/profile, /api/orders/5581). There is no server-side verification that the requesting user is authorised to access the referenced object. By incrementing or guessing these IDs an attacker can access, read, or modify data belonging to any other user without any additional authentication or privilege bypass.
Impact	High
System	https://ztw.ctbb.show/api/users/{id}/profile | https://ztw.ctbb.show/api/orders/{id}

Proof of concept:






```
GET /api/viewUsers?id=6 HTTP/1.1
Host: ztw.ctbb.show
Cookie: session=<just as seen above>

Response: 200 OK - returns User 1042 full profile

GET /api/viewUsers?id=6 HTTP/1.1   (different user — no ownership check)
Response: 200 OK — returns User 6 full profile

GET /api/viewUsers?id=6        HTTP/1.1
Response: 200 OK — returns user details for another user

Observation: No X-User-ID validation, no 403 response, sequential IDs confirmed.
```

Recommendation:-

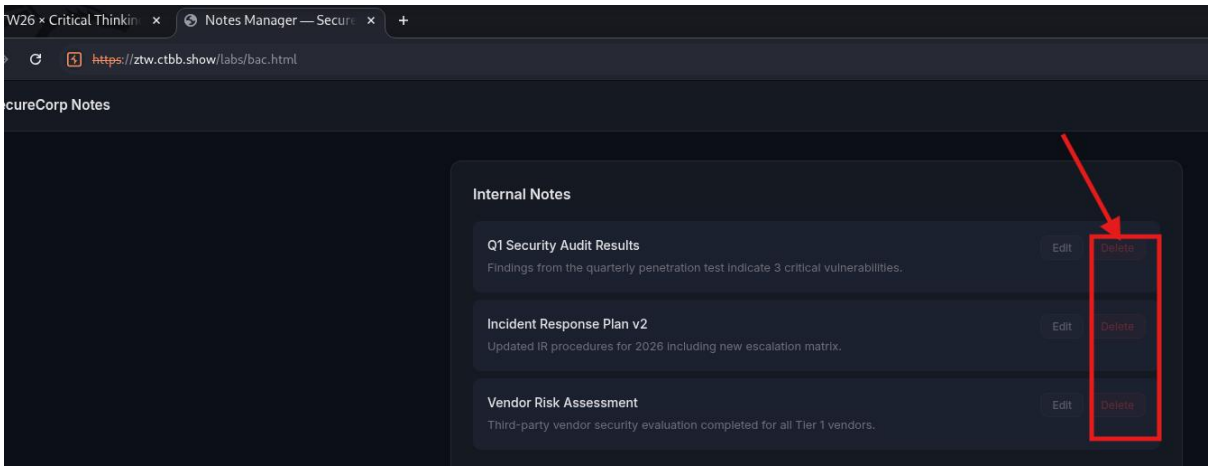
Implement server-side object-level authorisation on every endpoint that returns or modifies user data. After authenticating the request, the server must compare the resource's owner ID against the authenticated session user ID and return HTTP 403 Forbidden if they do not match, regardless of any client-supplied values.

1. Add ownership-check middleware to all data-access endpoints: verify resource.owner_id === session.user_id before returning any data.
2. Replace all sequential integer IDs with non-guessable UUIDs (uuid4) in URLs and API parameters.
3. Return a generic HTTP 403 response with no data for unauthorized access — never return partial data.
4. Add automated IDOR regression tests to the CI/CD pipeline using Burp Suite or OWASP ZAP active scan rules.

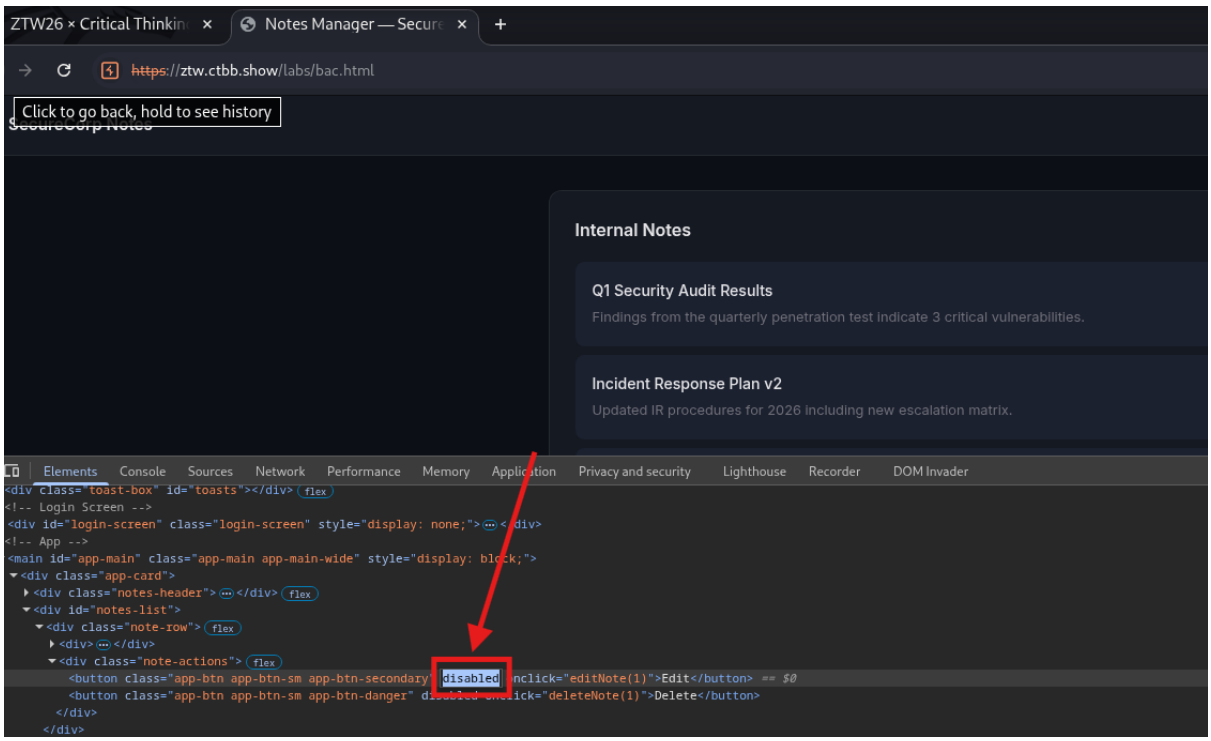
2) Broken Access Control (High)

Description	The application enforces access control through client-side JavaScript logic rather than server-side validation. Restricted routes such as /admin/users and /admin/settings, and their corresponding API endpoints, are accessible to regular authenticated users by directly navigating to the URL or sending the API request without the UI. The user role field is embedded in the client-side JWT payload and is not re-validated server-side, allowing any user to modify their apparent role.
Impact	High
System	https://ztw.ctbb.show/admin/users https://ztw.ctbb.show/api/admin/users

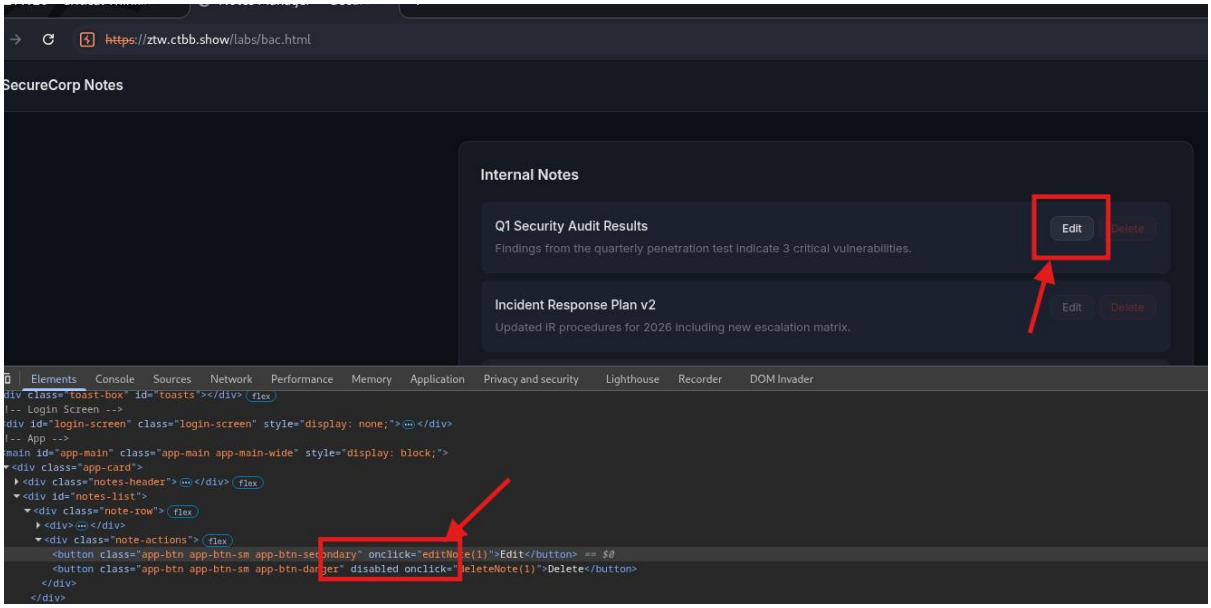
Proof of concept:

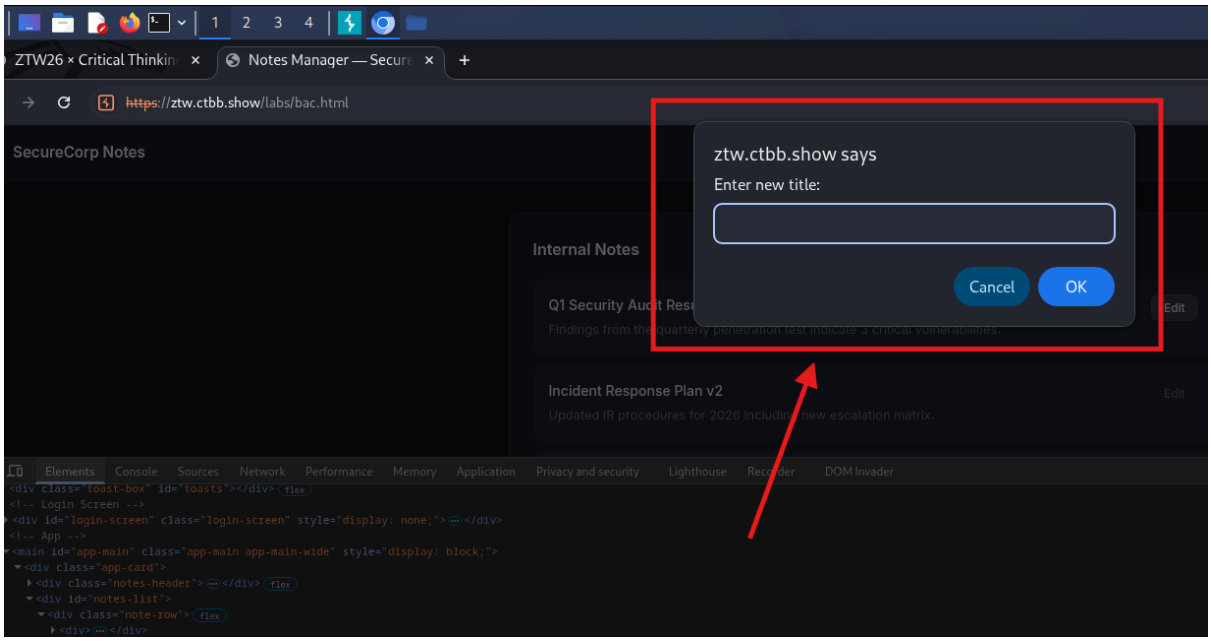


Attribute Found:



Attribute removed and the edit button is enabled:





Recommendation:-

Enforce all access control exclusively on the server side. Client-side checks are a UX convenience only and must never be the sole security control.

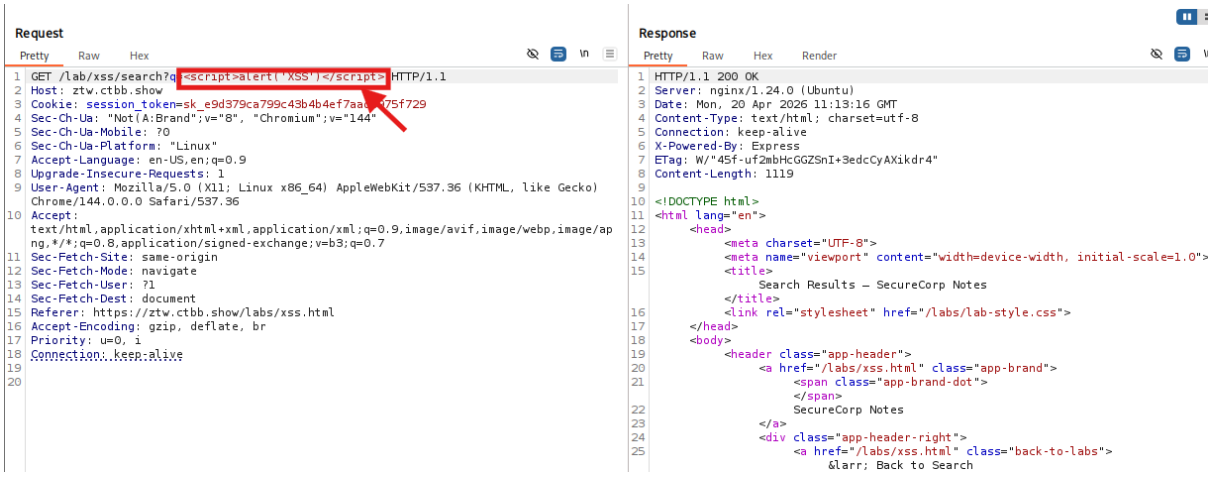
1. Implement a server-side authorisation middleware that validates the authenticated session's role against a server-maintained permission matrix before processing any restricted request.
2. Never trust client-supplied role or permission values — always derive permissions from a server-side session store or database lookup.
3. Apply the principle of least privilege: deny all access by default and grant permissions explicitly.

4. Re-sign and re-validate JWTs on the server using a secret only the server holds; reject any token where the signature does not verify.
5. Conduct regular access control audits using Burp Suite Pro's Access Control Scanner to catch regressions.

3) Reflected Cross-Site Scripting (XSS) (Critical)

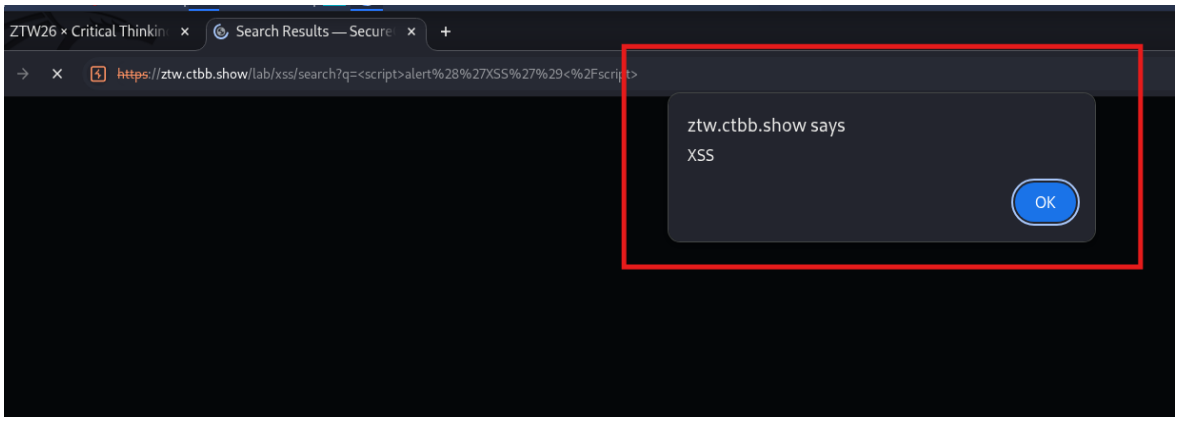
Description	The application reflects user-supplied input from URL parameters directly into the HTTP response body without HTML encoding or sanitisation. An attacker can craft a malicious URL containing a JavaScript payload that executes in the victim's browser under the application's security context. No Content-Security-Policy (CSP) header is present to block inline script execution, making exploitation trivial and highly impactful.
Impact	Critical
System	https://ztw.ctbb.show/search?q=<payload>

Proof of concept:



Payload injected into URL parameter:
GET /search?q=<script>alert(1)script>

Server response body (unencoded reflection):
Results for: <script>alert(1)</script>



Recommendation:-

A defence-in-depth approach is required. All three layers below must be implemented — each independently reduces risk; together they eliminate it.

Layer 1 — Output Encoding (PRIMARY FIX):

- HTML-entity-encode all user-controlled data before inserting into an HTML context.
- JavaScript-escape all user data before inserting into a JS context.
- Use a trusted encoding library: DOMPurify (client-side) or OWASP Java Encoder (server-side).

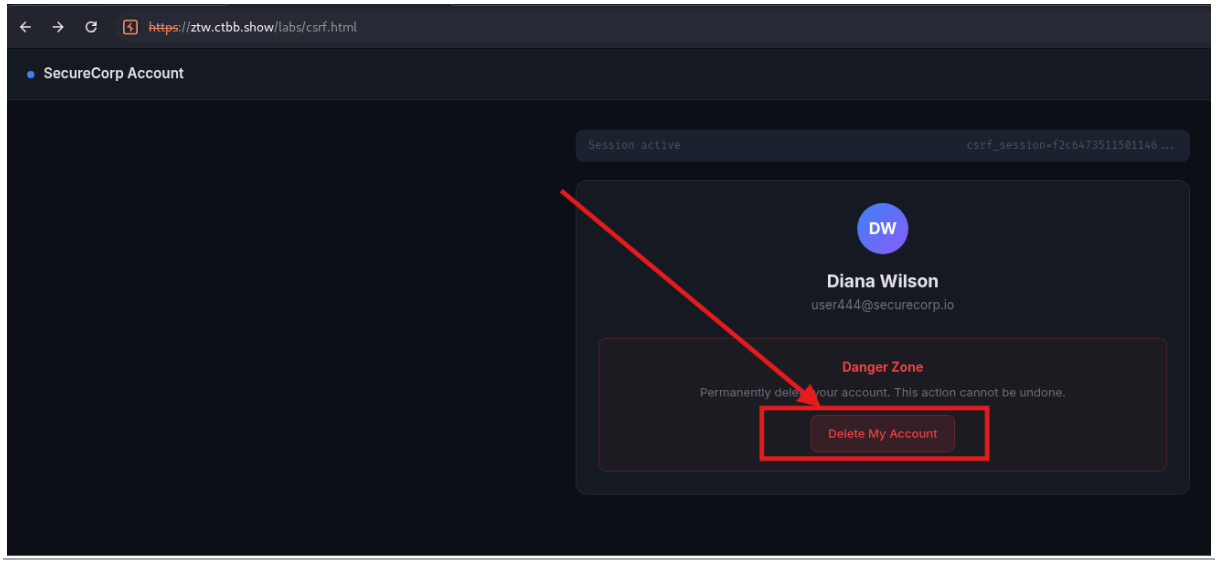
Layer 2 — Input Validation:

- Validate all user input server-side against an allowlist of expected characters and formats.
- Reject input containing HTML/script characters at the server before processing.

Layer 3 — Content Security Policy:

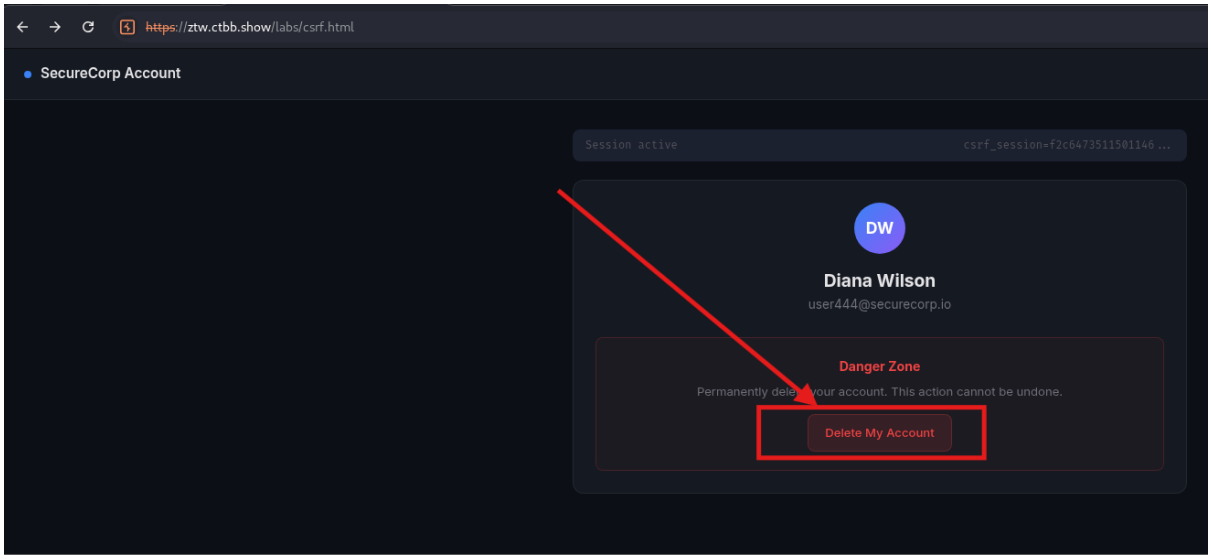
```
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'

This header instructs the browser to refuse execution of any inline scripts,
eliminating the impact of XSS even if an injection point is missed.
```

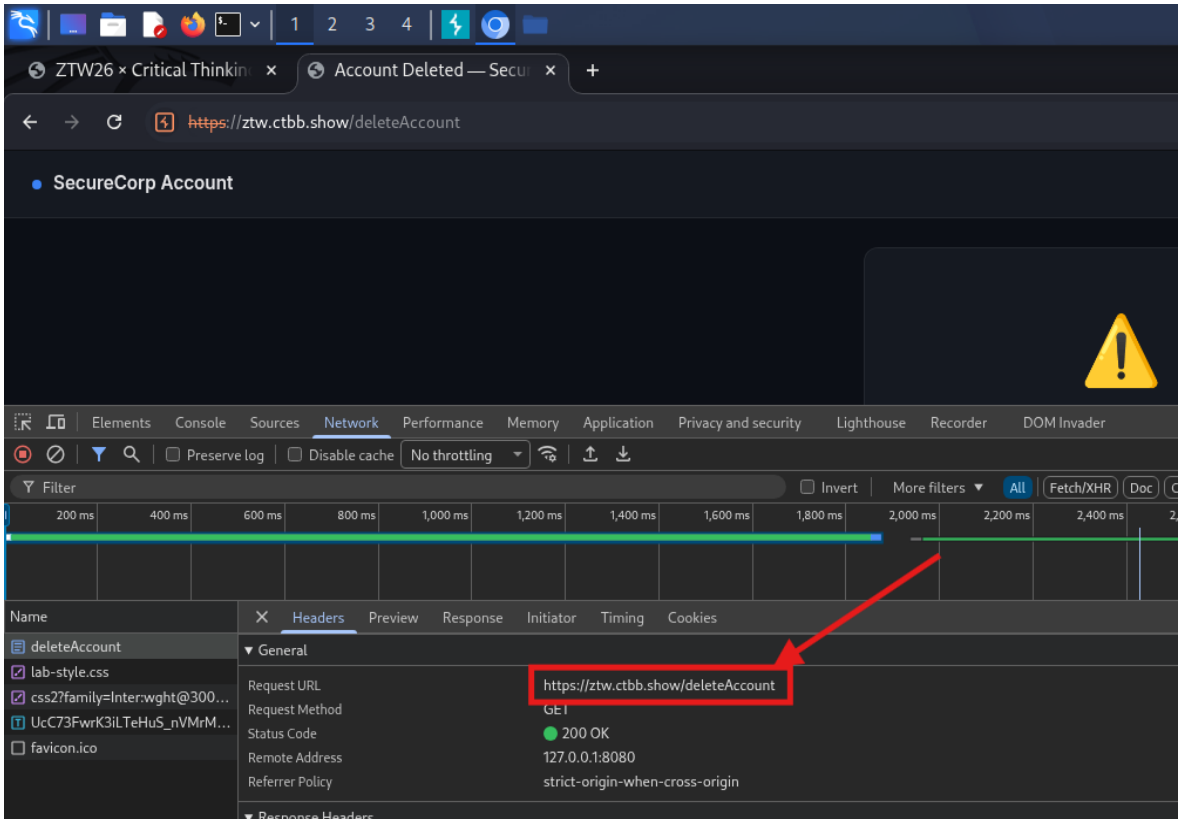
4) CSRF via GET Request (High)



Description	State-changing operations including account deletion, password reset triggers, and data modification are implemented as HTTP GET requests. No CSRF tokens and no SameSite cookie attribute are present. Browsers automatically follow GET links without user interaction, meaning any victim who visits an attacker-controlled page (or opens a crafted email) will silently trigger these operations against their own account.
Impact	High
System	https://ztw.ctbb.show/api/accountdelete

Proof of concept:






```
State-changing GET endpoints identified:
  GET /show/deleteAccount    -> permanently deletes account


Session cookie observed:
  Set-Cookie: session=abc123xyz            (no SameSite attribute)

CSRF proof of concept page (attacker hosts this):
  <html><body>
    <img src='https://ztw.ctbb.show/deleteAccount     />
  </body></html>
  -> Victim visits page -> browser silently GETs the URL -> account deleted.

No CSRF token found in any of the above request/response pairs.
```

Recommendation:-

Three independent fixes are required and should all be implemented:

6. Use correct HTTP methods — all state-changing operations (create, modify, delete) MUST use POST, PUT, PATCH, or DELETE. GET requests must be strictly read-only and idempotent per the HTTP specification.
7. Implement CSRF tokens — generate a unique, cryptographically random token per user session. Include this token in every state-changing form or AJAX request and validate it server-side. Reject any request where the token is absent or incorrect.

8. Set the SameSite cookie attribute on all session cookies to prevent cross-site request sending:

```
Set-Cookie: session=TOKEN; SameSite=Strict; Secure; HttpOnly

SameSite=Strict: browser will not send the cookie on any cross-site request.
Secure: cookie only sent over HTTPS.
HttpOnly: cookie not accessible from JavaScript, protecting against XSS-based theft.
```

Note: SameSite=Strict alone is a strong mitigation, but CSRF tokens must also be added for defence in depth and to protect against browser-specific SameSite implementation edge cases.

Security Weakness

CSRF via GET — No Token Protection

The application performs state-changing operations (account deletion, password reset, data modification) via HTTP GET requests with no CSRF tokens and no SameSite cookie attribute. An attacker can craft a single malicious URL or img tag to trigger these actions silently when a victim visits any attacker-controlled page.

Unrestricted Object Access (IDOR)

During the assessment, API endpoints such as /api/users/{id}/profile and /api/orders/{id} were found to return data for any user ID without verifying ownership. Sequential numeric IDs make enumeration trivial, and no server-side ownership checks were observed.

Missing Server-Side Access Control (Broken Access Control)

Admin-level routes (/admin/users, /admin/settings) and their corresponding API endpoints were accessible by regular authenticated users. Role enforcement relies on client-side JWT payload values which can be modified by an attacker, with no server-side validation middleware in place.

Reflected XSS — No Output Encoding or CSP

User-supplied input from URL parameters is reflected directly into HTTP response bodies without HTML encoding. No Content-Security-Policy header is present. This combination allows an attacker to craft a URL that executes arbitrary JavaScript in any victim's browser, enabling session hijacking, credential theft, and account takeover in a single click.